

Department of Electrical and Information Engineering

University of Ruhuna

Final Report

Cyber AI Agent: Multi-Model Intrusion Detection and Explanation Pipeline

Advanced Artificial Intelligence Group Project

Student Name 1 Index No.

Student Name 2 Index No.

Student Name 3 Index No.

30/05/2026

Department of Electrical and Information Engineering

University of Ruhuna

Contents

1	Executive Summary	4
2	Problem Statement & Motivation	4
3	Literature Review	4
4	Methodology	5
4.1	Overall Pipeline	5
4.2	Technique Mapping	5
4.3	Preprocessing Strategy	5
4.4	Model Design	6
4.5	Fusion and Explanation	6
5	Implementation Details	6
5.1	Software Architecture	6
5.2	Backend Layer Structure	6
5.3	Notebook Outputs	7
6	Experimental Setup & Results	7
7	Evaluation & Performance Metrics	8
8	Discussion	8
9	Conclusion & Future Work	9
10	References	9
11	Appendices	9
11.1	Project File Structure	9
11.2	Technologies Used	10

List of Figures

1	Cyber AI Agent software architecture.	6
2	Core project structure	9

List of Tables

1	Mapping of Assignment Techniques to the Project	5
2	Backend Layer Responsibilities	7
3	Notebook Workflow and Outputs	7
4	Recommended Results Table for Final Submission	8
5	Metrics to Report in the Final Version	8
6	Technologies Used in the Project	10

Executive Summary

Cyber AI Agent is a layered cybersecurity analytics system for detecting and explaining malicious network traffic. It ingests CSV flow records, preprocesses them into reusable artifacts, applies three complementary AI models, and fuses their outputs into a final threat verdict with a human-readable explanation. The project satisfies the assignment requirement to use multiple Advanced AI techniques by combining NLP, transformer-based transfer learning, and generative anomaly detection in one reproducible pipeline.

The system is implemented as a set of notebooks for training and evaluation and a FastAPI backend for runtime inference. The notebooks produce the scaler, label mappings, feature names, clipped values, trained models, and metrics used by the backend. The final service exposes upload, prediction, reporting, dashboard, and debug routes, enabling both demonstration and experimentation.

Problem Statement & Motivation

Network intrusion detection often suffers from class imbalance, noisy features, and limited interpretability. Traditional detectors can identify attacks, but they do not explain why a record was flagged or how different models agree with each other. This project addresses those issues by combining a tabular classifier, a transformer-based text classifier, and an autoencoder-based anomaly detector, then adding an explanation layer on top.

The motivation is practical: a security team needs a system that can inspect flow logs quickly, identify suspicious traffic, reduce false negatives on rare attacks, and produce concise explanations that support incident response. The selected domain is Telecommunications, which is one of the recommended domains in the assignment and is well matched to network flow analysis.

Literature Review

This project builds on three established ideas in modern AI systems. First, supervised tabular classifiers such as gradient-boosted decision trees remain strong baselines for structured traffic features because they handle heterogeneous feature scales and class imbalance effectively. Second, transformer-based models such as BERT have demonstrated strong transfer-learning performance in many classification tasks and can be adapted here by converting each flow record into a natural-language description. Third, autoencoders are widely used for anomaly detection because they learn a compact representation of normal behavior and assign high reconstruction error to unusual inputs.

Recent applied cybersecurity systems increasingly combine multiple model families rather than relying on a single detector. This project follows that direction by using model diversity as a strength: XGBoost captures tabular structure, BERT captures semantic descriptions, and the autoencoder detects unusual patterns that may not be well separated in the supervised labels.

The LLM explanation layer further improves usability by translating technical detections into analyst-friendly summaries.

Methodology

Overall Pipeline

The methodology follows a five-stage pipeline.

1. **Data ingestion:** CSV flow records are parsed, validated, and enriched with IP metadata.
2. **Preprocessing:** The selected 18 features are cleaned, standardized, clipped, and saved with the fitted scaler and feature metadata.
3. **Model training:** XGBoost, BERT, and a denoising autoencoder are trained independently using the notebook workflow.
4. **Fusion:** The model outputs are combined using a weighted decision layer to produce the final attack label.
5. **Explanation:** The top detected threats are passed to an LLM explainer that returns structured JSON narratives.

Technique Mapping

Table 1: Mapping of Assignment Techniques to the Project

Technique	How it is used
NLP	Flow records are converted into natural-language descriptions before BERT classification.
Transformer-based model	BERT is used for traffic-text classification.
Generative AI	A denoising autoencoder models benign traffic and flags anomalies via reconstruction error.
Transfer learning	BERT leverages pre-trained transformer weights rather than training from scratch.
Prompt engineering	The LLM explainer uses a structured JSON prompt for analyst-facing threat summaries.

Preprocessing Strategy

The preprocessing notebook selects 18 core network-flow features, removes duplicates, fills missing values, handles infinite values, applies standard scaling, and clips extreme scaled values at the 99th percentile per feature. Persisting both the scaler and clip values is important because the autoencoder and ensemble notebooks must consume the same transformed feature distribution during inference.

Model Design

XGBoost is used as the primary structured-data classifier and is trained with class-balanced sample weights to reduce bias toward dominant classes. BERT receives text generated from the same flow features and classifies attack types from a semantic representation. The autoencoder is trained only on benign samples, which lets it learn a compact normal-behavior manifold and treat high reconstruction error as anomaly evidence.

Fusion and Explanation

The ensemble layer combines model confidences using weighted fusion and then applies a threshold to suppress low-confidence alarms. The backend then enriches detected threats with external context and generates human-readable explanations. In environments where API credentials are unavailable, the LLM explainer falls back to mock mode so the demo remains runnable.

Implementation Details

Software Architecture

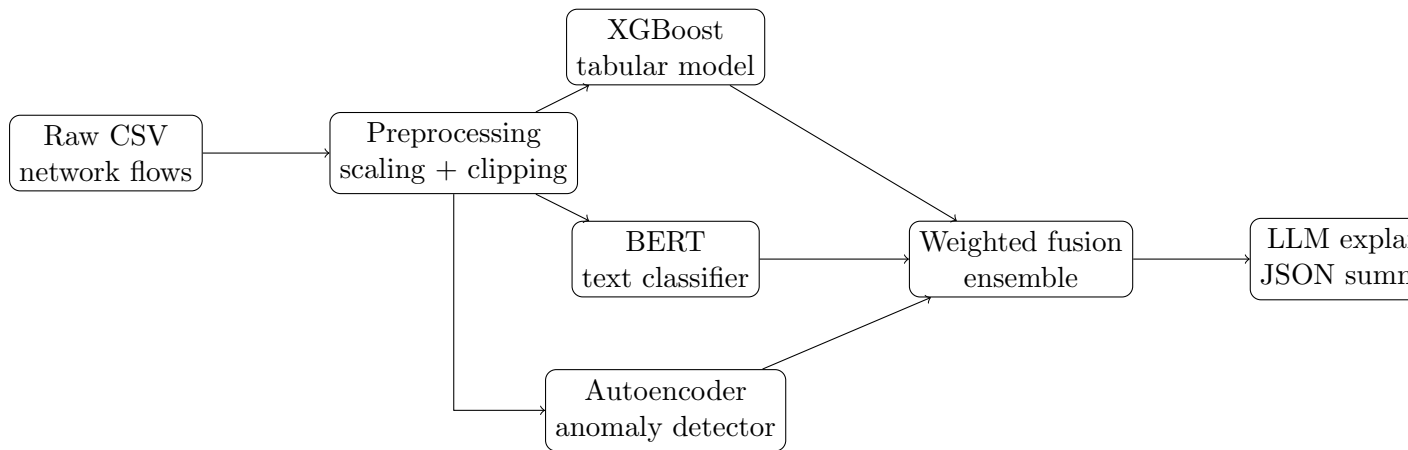


Figure 1: Cyber AI Agent software architecture.

Backend Layer Structure

The FastAPI application loads all model layers at startup and routes each request through the following sequence: ingestion, preprocessing, XGBoost, BERT, autoencoder, fusion, MCP enrichment, and LLM explanation. This keeps training and inference decoupled while preserving the same transformation logic across notebooks and runtime services.

Table 2: Backend Layer Responsibilities

Layer	Component	Responsibility
1	Ingestion	Parse CSV input, validate columns, and extract IP metadata.
2	Preprocessing	Align feature order, clean values, scale and clip data, generate text.
3	XGBoost	Predict attack labels from tabular features.
4	BERT	Predict attack labels from generated textual descriptions.
5	Autoencoder	Flag anomalies from reconstruction error.
6	Fusion	Combine model outputs using weighted decision logic.
7	MCP tools	Enrich detections with extra threat context.
8	LLM explainer	Produce concise human-readable incident summaries.

Notebook Outputs

Table 3: Notebook Workflow and Outputs

Notebook	Purpose
<code>01_preprocessing.ipynb</code>	Clean data, scale features, clip outliers, save artifacts
<code>02_xgboost.ipynb</code>	Train the supervised tree model and save feature importance
<code>03_bert.ipynb</code>	Build textual descriptions and fine-tune the transformer classifier
<code>04_autoencoder.ipynb</code>	Train the benign-only denoising autoencoder and tune thresholds
<code>05_ensemble.ipynb</code>	Fuse predictions, compare ablations, and export final reports

Experimental Setup & Results

The experiments are designed to compare each model individually and then as a combined ensemble. XGBoost and BERT are evaluated using multiclass accuracy, precision, recall, and F1-score. The autoencoder is evaluated as a binary anomaly detector using detection rate and false positive rate. The ensemble notebook additionally produces per-class F1 comparisons and ablation studies.

Table 4: Recommended Results Table for Final Submission

Model	Accuracy	F1 Score	Notes
XGBoost	–	–	Tabular baseline
BERT	–	–	Transformer-based classifier
Autoencoder	–	–	Binary anomaly detector
Fusion ensemble	–	–	Final system output

The final notebook outputs should be inserted into this section after the models are executed. The project already saves the relevant metrics JSON files and plots, so the report can be updated directly from the notebook outputs once the final training run is complete.

Evaluation & Performance Metrics

The evaluation should cover both quantitative and qualitative analysis. For the supervised classifiers, report weighted accuracy, precision, recall, and F1-score. For the autoencoder, report reconstruction error statistics, the selected anomaly threshold, detection rate, and false positive rate. For the ensemble, report overall F1-score improvements over individual models, per-class F1 changes, and ablation-study findings.

Table 5: Metrics to Report in the Final Version

Metric	Why it matters
Accuracy	Overall correctness of predictions
Precision	False alarm control
Recall	Attack detection sensitivity
F1-score	Balance between precision and recall
Detection rate	How many anomalies are caught
False positive rate	Operational cost of false alarms

Discussion

The main strength of the project is its diversity of model families. The tabular classifier captures structured flow patterns, the transformer uses semantic text representations, and the autoencoder provides an unsupervised safety net for unusual traffic. The ensemble improves robustness by preventing any one model from dominating the final decision.

The main limitation is environment sensitivity on Windows, especially for TensorFlow and transformer imports. Another practical limitation is that the LLM explanation layer depends on valid API credentials unless mock mode is used. Finally, report numbers should be regenerated after the final notebook run so the written analysis matches the saved metrics.

Conclusion & Future Work

Cyber AI Agent demonstrates a complete Advanced Artificial Intelligence project that combines NLP, transformer-based learning, generative anomaly detection, prompt-driven explanation generation, and ensemble fusion. The system is practical for a live demo and is structured so that each notebook can be rerun independently.

Future work should include packaging the notebooks into a single reproducible pipeline, adding automated smoke tests for artifact loading, integrating a front-end dashboard for analysts, and calibrating the fusion layer on a held-out validation set. A larger curated dataset and additional class balancing strategies could further improve rare-attack detection.

References

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*.
- [3] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Appendices

Project File Structure

```
1 cyber-ai-agent/  
2 |-- backend/  
3 |   |-- app/  
4 |     |-- main.py  
5 |     |-- pipeline.py  
6 |     |-- layers/  
7 |       |-- ingestion.py  
8 |       |-- preprocessing.py  
9 |       |-- xgboost_model.py  
10 |      |-- bert_model.py  
11 |      |-- autoencoder_model.py  
12 |      |-- fusion.py  
13 |      |-- mcp_tools.py  
14 |      +-- llm_explainer.py
```

```

15 | | |-- routes/
16 | | +-- utils/
17 | |-- datasets/
18 | |-- trained_models/
19 | +-- notebooks/
20 |     |-- 01_preprocessing.ipynb
21 |     |-- 02_xgboost.ipynb
22 |     |-- 03_bert.ipynb
23 |     |-- 04_autoencoder.ipynb
24 |     +-- 05_ensemble.ipynb
25 |-- frontend/
26 |-- reports/

```

Figure 2: Core project structure

Technologies Used

Table 6: Technologies Used in the Project

Technology	Role
FastAPI	Backend API layer
XGBoost	Structured tabular classification
Transformers / BERT	NLP-based traffic classification
TensorFlow / Keras	Autoencoder anomaly detection
PyTorch	Transformer runtime support
Groq API	LLM-based explanations
Pandas / NumPy	Data preprocessing and analysis
Joblib	Artifact persistence